实 验 报 告

实验名称：实验二感知器模型实现并进行分类

课程名称：　认知科学与类脑识别

实验地点：　　K2

实验日期：　2024.9.27

班　级：　2022 级人工智能班

姓　名：　　薛润泽

学　号：　202200130077

一 实验目的：

加深对感知器模型的理解，能够使用感知器模型解决简单的分类问题

二 实验环境：

Windows，python

三 实验内容：

感知器是一种二元线性分类器。感知器通过对输入特征进行线性组合来进行分类。其基本结构包括输入层和输出层。输入层接收外部信息，每个输入节点接收一个输入信号；输出层则处理这些信息并输出结果

四 实验步骤：

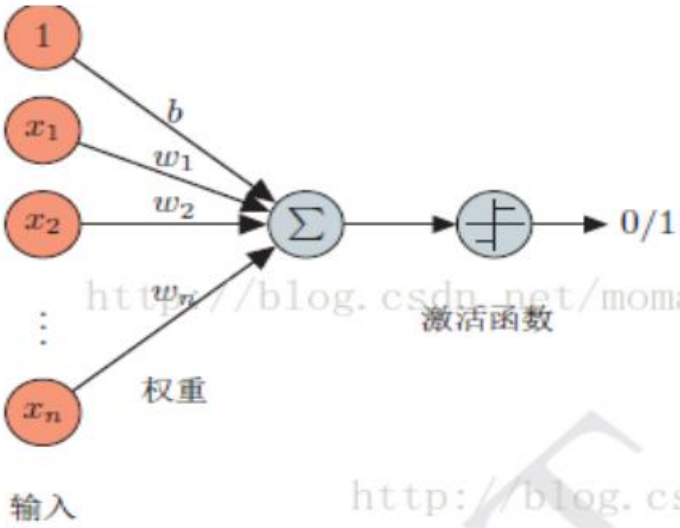
实验程序：

（1）构建出与（and），或（or），非（not）真值表，其中包含训练输入数据和对应输出数据。

```
train_data_and_in = np.array([[0, 0],[0, 1],[1, 0],[1, 1]]) #4行2列向量
train_data_and_out = np.array([0, 0, 0, 1])
train_data_or_in = np.array([[0, 0],[0, 1],[1, 0],[1, 1]])
train_data_or_out = np.array([0, 1, 1, 1])
train_data_not_in = np.array([[0], [1]])
train_data_not_out = np.array([1, 0])
```

（2）构建感知机类

感知器模型如下：



http://blog.csdn.net/momaoj
http://blog.csdn.

基本数学模型为 Y = WX + b

其中，通过输入 n 维数据 x（b 单独运算），与对应 n 维的 w 做点积后再与 b 加和。将加和结果通过激活函数（ReLU 函数），大于 0 则为 1，小于等于 0 则为 0，得出 0/1 二分类结果。

构建感知机类，其中包含__init__()初始化函数，activation()以 ReLU 函数为基础的阶跃函数，train()训练函数，pre()预测函数。

其中，train()训练函数的权重更新规则如下：

权重更新规则如下所示：

$$w_i \leftarrow w_i + \Delta w_i$$
$$b \leftarrow b + \Delta b$$

$$\Delta w_i = \eta(t - y)x_i$$
$$\Delta b = \eta(t - y)$$

其中 w 是构建的权重向量，b 是偏置值，η 是训练步长，

t 是输入对应的实际输出，y 是对应的预测输出。

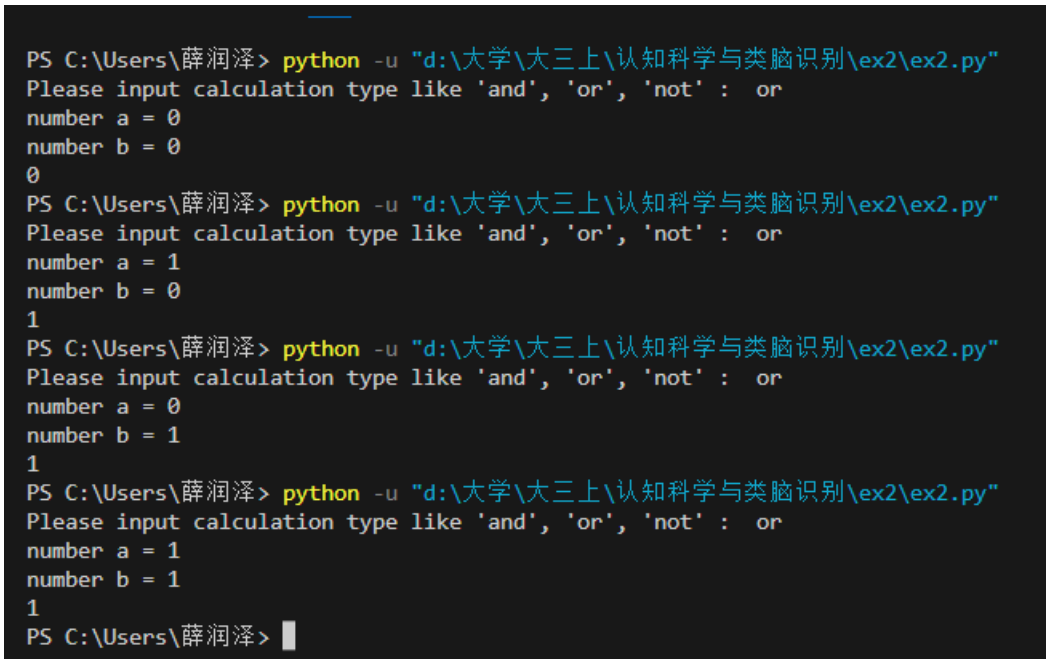
建立 model_and，model_or，model_not 等不同的模型。使用相应数据进行训练，然后进行预测。

1. 程序实验结果：

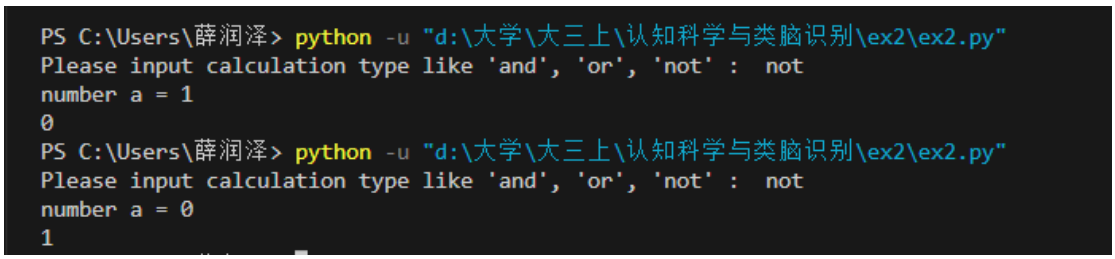
（1）选择 and，分别进行[0, 0]，[1, 0]，[0, 1]，[1, 1]的 and 运算，结果如下：

```
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex2\ex2.py"
Please input calculation type like 'and', 'or', 'not' :  and
number a = 0
number b = 0
0
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex2\ex2.py"
Please input calculation type like 'and', 'or', 'not' :  and
number a = 0
number b = 1
0
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex2\ex2.py"
Please input calculation type like 'and', 'or', 'not' :  and
number a = 1
number b = 0
0
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex2\ex2.py"
Please input calculation type like 'and', 'or', 'not' :  and
number a = 1
number b = 1
1
```

（2）选择 or，分别进行[0, 0]，[1, 0]，[0, 1]，[1, 1]的 or 运算，结果如下：



（3）选择 not，进行 0 和 1 的 not 运算，结果如下：



2. 分析和改进：

本次实验代码大多自己写，较少使用 python 库中内容，可以使用 NumPy 库中内容来帮助开发，降低开发时间。

NumPy 有内置的激活函数，可以调用其内置激活函数。

处理 and，or，not 操作的代码较为重复，可以改进降低代码量。

五 实验小结：

该实验实现了用于线性二元分类的感知器模型，通过参照与或非等操作建立真值表，分别构建了与、或、非三个感知器模型，用于 0,1 输出二元分类。

可以发现，与、或、非三种操作都是线性操作，所以线性二元分类的感知器模型可以精准的分类。但是异或运算是非线性操作，线性二元分类的感知器模型对异或运算无能为力。

六、代码附录：

```
import numpy as np

train_data_and_in = np.array([[0, 0],[0, 1],[1, 0],[1, 1]]) #4
行 2 列向量
train_data_and_out = np.array([0, 0, 0, 1])
train_data_or_in = np.array([[0, 0],[0, 1],[1, 0],[1, 1]])
train_data_or_out = np.array([0, 1, 1, 1])
train_data_not_in = np.array([[0], [1]])
train_data_not_out = np.array([1, 0])

# y = wx
class predictor:
    def __init__(self, w_size, step_length): # 输入参数列表大小和
训练步长，b 单独列出
        self.w = np.zeros(w_size) # 参数向量，列向量
        self.sl = step_length
        self.b = 0
    def activation(self, a): #使用 LeRU 函数
        if a <= 0:
            return 0
        else :
```

```
            return 1
    def train(self, x, y, epochs):# 输入原始 x,原始 y,训练论述 epochs
        for i in range(epochs):
            for j in range(len(x)):
                pre_y = self.activation(np.dot(self.w, x[j]) +
self.b)
                for k in range(len(self.w)):
                    delt_b = self.sl *(y[j] - pre_y)
                    delt_w = delt_b * x[j][k]
                    self.b = self.b + delt_b
                    self.w[k] = self.w[k] + delt_w
    def pre(self, x):
        re = np.dot(self.w, x) + self.b
        return self.activation(re)

ct = input("Please input calculation type like 'and', 'or',
'not' :  ")
if(ct == 'and'):

    model_and = predictor(2, 0.01)
    model_and.train(train_data_and_in, train_data_and_out, 1000)
    a = float(input("number a = "))
    b = float(input("number b = "))
    # input_x =
    input_vector = np.array([a, b])
    and_res = model_and.pre(input_vector)
    print(and_res)

elif(ct == 'or'):
    model_or = predictor(2, 0.01)
    model_or.train(train_data_or_in, train_data_or_out, 1000)
    a = float(input("number a = "))
    b = float(input("number b = "))
    input_vector = np.array([a, b])
    or_res = model_or.pre(input_vector)
```

```
    print(or_res)
elif(ct == 'not'):
    model_not = predictor(1, 0.01)
    model_not.train(train_data_not_in, train_data_not_out, 1000)
    a = float(input("number a = "))
    input_vector = np.array([a])
    not_res = model_not.pre(input_vector)
    print(not_res)
else:
    print("not such type")
```